

SismoKaos Operations Manual

How to perform each operation in `cnn_earthquake` : acquisition, windowing, training, evaluation and reporting. Every command here was read off the live interface, not from memory.

commit 4374150 entry point sk (21 commands) package src/sismokaos/

START

- 0 · Setup
- 1 · Which data source

ACQUIRE

- 2 · FDSN pull (KO)
- 3 · TDVMS campaign (TU)
- 4 · Rebuild the catalogue
- 5 · Choose stations

PREPARE

- 6 · Cut windows
- 7 · Repair a manifest

MODEL

- 8 · Train
- 9 · Evaluate

STUDY

- 10 · Continuous false alarms

FINISH

- 11 · Reports and figures
 - 12 · What happened
 - 13 · Reproduce a result
 - 14 · Gotchas
-

OP 0 Setup

One installed package, `sismokaos` . There is no `scripts/` directory any more; every tool is a module and `sk` is the only entry point.

```
# first time, or after any move of the checkout
uv sync                # creates .venv/, installs the package editable
direnv allow           # if you use direnv
uv run sk              # the command listing
uv run pytest -q        # 1040 passed, 1 skipped
```

After moving or copying the checkout, delete `.venv` first. `rm -rf .venv && uv sync`. A *copy* is the dangerous case – the editable `.pth` keeps pointing at the old `src`, so imports silently resolve to the stale checkout and you debug the wrong code.

Every tool also runs standalone, which is what you want inside a script or a `nohup`:

```
uv run python -m sismokaos.acquisition.afad_campaign status
uv run python -m sismokaos.continuous.cli report --help
```

Two machines. Work is developed locally and run on `vegs`, which holds the archives (`afad_raw/`, 47 GB), the datasets and the GPU. Keep both at the same commit – `git pull --ff-only` on `vegs` before running anything, or you will debug a difference that is only a stale checkout. If `vegs` is asleep: `ssh vegsjumper 'bash ~/wol_vegs.sh'`, it wakes in about 30 s.

OP 1 Deciding where waveforms come from

The choice is by **network**, not by `event-vs-continuous`. Getting this wrong costs days.

NETWORK	ROUTE	COST	USE IT FOR
KO (Kandilli)	<code>sk fdsn</code>	24 h in ~13 s; direct HTTP, no queue	anything on KO – training enrichment, more events, continuous spans
TU (AFAD)	<code>sk campaign + sk poll</code>	~2 min per station-day, email queue, whole days only	TU only – no FDSN node serves it

FDSN is roughly a hundred times cheaper and returns the exact window rather than a whole day. **TU's only advantage is station-disjointness:** 156 of the 163 FDSN KO stations are already in the training corpus, so FDSN cannot supply unseen stations and TU can.

One TDVMS request returns one station. The payload accepts a `stations` array and the reply says "the station(s) you requested", but only one arrives – measured 2026-09-06. N stations cost N

queue slots. Do not plan around batching.

OP 2 Pull event waveforms from FDSN

```
sk fdsn plan --catalog catalogs/catalog_current.csv \  
             [--network K0] [--url URL] > requests.csv  
  
sk fdsn fetch --requests requests.csv --out-dir raw/fdsn_magnitude \  
             [--url URL]
```

Produces one mseed per event-station under `--out-dir/<station>/event_<id>_raw.mseed` .

Noise windows for the same stations, which magnitude training needs as a negative pool:

```
sk fdsn-noise --requests requests.csv --event-dir raw/fdsn_magnitude \  
             --out noise_requests.csv [--per-station N] [--offset SEC]
```

The filename carries the event id, and a consumer parses it. `event_<id>_raw.mseed` with the station in the *path*, never in the filename. A previous build wrote `event_<id>_<station>_raw.mseed` and the non-greedy parse read the event id as `153534_TASB` , costing a full 13,150-window encode.

OP 3 Run a TDVMS campaign

A queue, not a download. You submit one window per plus-address, wait for mail, paste the link back, and refill the freed slot.

Credentials

```
# environment only — .env* are gitignored and chmod 600  
AFAD_IMAP_HOST=... AFAD_IMAP_USER=... AFAD_IMAP_PASS=...
```

The loop

```
sk campaign --ledger mant.jsonl plan --station MANT \  
           [--start 2024-05-01] [--end 2026-08-10]  
  
sk campaign --ledger mant.jsonl next --email you+a1@example.com
```

```
# or fill every free slot at once, then print where the ledger stands:
sk campaign --ledger mant.jsonl pump --email you+a1@x --email you+a2@x

sk poll --ledger mant.jsonl --out-dir afad_raw \
  --search '(UNSEEN FROM "tdvms@afad.gov.tr")' --pump

# manual paths, when the poller is not running
sk campaign --ledger mant.jsonl paste --url https://tdvms.../x.zip
sk campaign --ledger mant.jsonl mark --email you+a1@x --state nodata
sk campaign --ledger mant.jsonl reset --start 2024-05-01
sk campaign --ledger mant.jsonl status
```

Produces chunk archives at `afad_raw/<STATION>/<STATION>_<date>.zip` , roughly 21 days each.

Why plus-addresses. TDVMS keys its one-request-at-a-time limit on the literal address string, so `you+a1@x` and `you+a2@x` are separate queue slots. Several pollers can share one mailbox: each leaves mail addressed to a slot its own ledger never submitted from *unread*, for the poller that owns it.

Never chain submission onto a monitor or a foreground command. A 15-minute monitor timeout once killed the refill before it ran and the campaign sat idle with zero requests in flight, waiting for mail that was never coming. `pump` is safe to re-run — `next` refuses to double-claim.

Being listed is not being served. `https://tdvms.afad.gov.tr/api/Data/GetStations` lists 390 TU stations for free, but a listed station can still return `nodata` at every date. Probe before committing queue slots — two stations once sat `claimed` for an afternoon holding slots because an unlistable station was discovered only after the ledger was written.

OP 4 Rebuild the event catalogue

```
sk catalog --out catalogs/catalog_afad_$(date +%F).csv \
  [--start 2000-01-01] [--end 2026-12-31] [--min-magnitude 1.5] \
  [--box MINLAT MAXLAT MINLON MAXLON] [--page-days N] [--retries N]
```

Produces a catalogue CSV with `EventID`, `Date`, `Latitude`, `Longitude`, `Depth`, `Magnitude`, `Location` .
Point `catalogs/catalog_current.csv` at it as a symlink.

The AFAD API is public, unauthenticated and unthrottled — a full 413,785-event rebuild takes under 30 s and has nothing to do with the TDVMS waveform queue.

The catalogue in this repo was never KOERI's. All three older local catalogues were AFAD data despite being labelled KOERI, and the one in use was missing 1,688 of 5,770 regional events including 253 at $M \geq 4.0$ – essentially the whole February 2025 Santorini–Amorgos swarm. Re-deriving labels moved a chaos-feature base rate from 25.1% to 39.9%. If you find a result that predates the 2026-08-30 rebuild, re-derive before trusting it. The *waveforms* genuinely are KOERI.

Parsing gotchas, both real: `pd.to_datetime(..., format="ISO8601")` is required because pre-2010 records omit fractional seconds, and the bounding box must reach lat 44.5 or six Black Sea events are clipped.

OP 5 Choose and characterise stations

```
# rank candidate stations by how much of the catalogue they cover
sk station-select --events catalogs/catalog_current.csv \
  --stations catalogs/istasyon_katalog.csv \
  [--network TU] [--radius 500] [--min-magnitude 2.0] [--top 40]

# per-event SNR at one station – needed before any recall claim
sk station-range --zips 'afad_raw/MANT/*.zip' --station MANT \
  --stations-csv catalogs/istasyon_katalog.csv \
  --catalog catalogs/catalog_current.csv --out mant_range.csv

# what a station's catalogue misses relative to a truth catalogue
sk station-loss --stations catalogs/istasyon_katalog.csv \
  --broken old_catalog.csv --truth catalogs/catalog_current.csv
```

station-range produces a per-event SNR table. This is not optional – it is what makes recall a statement about the detector rather than about the catalogue.

Only 27.5% of catalogued events reach SNR 3 at MANT (median 1.39). Score a detector against every catalogued event and the event AUC is 0.67–0.73 – that number describes the catalogue's reach, not the model. Always pass `--snr-csv` downstream and state the SNR cut beside any recall.

OP 6 Cut training windows

Catalogue-anchored (the usual case)

```
sk cut-events --zips 'afad_raw/MANT/*.zip' --station MANT \
  --stations-csv catalogs/istasyon_katalog.csv \
  --catalog catalogs/catalog_current.csv \
  --snr-csv mant_range.csv [--snr-min 3.0] \
  [--window-seconds 6 10 20] [--pre 2.0] [--fs 100] \
  --out-dir raw/continuous_windows
```

`--window-seconds` takes several lengths in one pass, which is how the 6 s / 10 s / 20 s corpora were cut over identical events – a controlled window-length experiment for free.

Alarm-anchored (what a cascade actually sees)

```
# first get alarm times from a detector arm (see OP 10)
sk falsealarm timing --scores 'scores_mant/6s/*.npz' --station MANT ... \
  --threshold 0.9971 --out mant_alarm_times.csv

sk cut-events ... --anchor-csv mant_alarm_times.csv \
  --anchor-column alarm_epoch [--anchor-lag 0.0] \
  --out-dir raw/alarm_windows
```

Produces `<out-dir>/<tag>/<station>/{eq,noise}/` . Re-cut existing windows to another length without re-reading archives: `sk cut-length --src DIR --seconds 10 --out DIR2` .

Record which arm and which threshold produced an anchor CSV. The existing `mant_alarm_times.csv` does not, so the dataset built from it cannot be compared with one cut from another detector – the provenance is unrecoverable and the cut has to be redone.

OP 7 Repair a manifest's distances

```
sk distances --manifest dataset_magreg_fdsn_10s/manifest.csv \
  --station-coords ../seismic_cli/catalogs/station_coords.csv \
  --catalog catalogs/catalog_current.csv \
  [--tolerance 5.0] [--report-above 1.0]           # dry run
sk distances ... --write                          # after reading what it says
```

Produces a repaired `distance_km` column and a `.pre-distance-repair` backup beside it.

Run this whenever a manifest has missing or suspect distances. It recomputes *every* row and refuses to write unless it reproduces the existing values within `--tolerance` – a recomputation that disagrees broadly is wrong, and writing it would swap a partial column for a confidently incorrect one.

Missing distance_km is not benign. The trainer mean-imputes NaN aux after standardizing, so every row from a station with no distance trains against one fixed wrong value – silently. It also breaks the `ridge(log_snr, log_distance)` floor every magnitude model is judged against. On the FDSN corpus 42 of 161 stations had none for any row.

OP 8 Train a model

Indexed by **what you are predicting**, not by architecture, because the trainers are named after their networks and that is the wrong axis to search along.

```
sk train                                # 25 tasks, grouped by what the label answers
sk train --predicts forecast             # one group
sk train detect --label                  # what this task's label actually is
sk train detect --help                   # the trainer's own flags

sk models                               # every architecture, by input shape
sk models dual-channel                   # one model's branches and defaults
sk models --spec trained_model_x        # the spec saved beside those checkpoints
```

A real run

```
sk train magnitude --dataset-dir dataset_magreg_fdsn_10s \
  --channels 1d+2d --split-by auto \
  --seed 42 --seed-split 0 \
  --save-dir trained_model_magreg_fdsn10s_nonaux_p0
```

The split flags, which decide what the number means

--SPLIT-BY	WHAT IT DOES	WHEN
auto	both when the corpus has >1 station, else event	the default; prints which it chose
event	the generator's own split, unchanged	only when one station makes anything else impossible
station	stations disjoint; a shared event can still leak	rarely — prefer both
both	station-disjoint AND event-disjoint	the honest protocol; costs ~85% of val/test on small corpora
...	...	...

<code>detector</code>	copies a detector's station partition	mandatory for cascade evaluation — see the trap
-----------------------	---------------------------------------	---

Produces checkpoints named with channels/split/seed/PID, a `model.json` recording the architecture, and a run record under `runs/` carrying `argv`, git commit, *whether the tree was dirty*, dataset identity, seeds and metrics.

A cascade's stage 2 must inherit stage 1's station partition. Without `--split-by detector --detector-manifest PATH`, 77% of detector test stations were regressor *training* stations, and the end-to-end number is inflated by site memorisation.

Vary `--seed-split`, not just `--seed`. Which stations land in test dominates: partition variance was measured at 2.4× model-seed variance. Three partitions gives an honest error bar; three seeds on one partition gives a flattering one.

Non-aux is the magnitude default (1d+2d). The aux vector is `(log_snr, log_distance)` and `log_distance` needs a hypocentre a fresh detection does not have, so `all` trains a model that cannot run in the cascade it exists for. Pass `--channels all` explicitly for an aux ablation.

OP 9 Evaluate a trained model

```
# where a magnitude regressor's error actually lives
sk magprofile --dataset-dir dataset_magreg_fdsn_10s \
  --ckpt trained_model_x/best_....pth \
  [--split-by both] [--seed-split 0] [--out profile.csv]

# end-to-end detector -> magnitude cascade
uv run python -m sismokaos.detection.cascade_eval \
  --detector-dir DS --detector-ckpt-dir CKPT \
  --magnitude-dir DS2 --magnitude-ckpt CKPT2 [--threshold T]
```

Both read `model.json` and the spec wins over the flags. They print the spec they used and name any flag that disagrees, so you no longer retype a checkpoint's geometry — and cannot silently score it as a different architecture.

Report MAE by magnitude band, always. An aggregate is largely a readout of the corpus's magnitude mix: on the 6 s corpus, 71.1% of test is $M \leq 2.5$ where MAE is 0.1430, against 0.4085 for $M > 3$. A corpus with more small earthquakes will look like a better model.

OP 10 The continuous false-alarm study

Five phases. The second is the expensive one and must not be redone to change a guard width, which is why they are separate.

```
# 1. the station's long-term (mu, sigma) – what every window is standardized against
sk falsealarm baseline --zips 'afad_raw/MANT/*.zip' \
  [--sample-chunks 6] --out mant_baseline.json

# 2. window and score the whole record; several arms share one read
sk falsealarm scan --zips 'afad_raw/MANT/*.zip' \
  --baseline-json mant_baseline.json \
  --arm 6s:6.0:trained_model_branch1d_asinh:cnn-lstm \
  --arm ponly:3.4:trained_model_ponly_matched:cnn-lstm \
  --out-dir scores_mant [--workers 6] [--limit-chunks N]

# 3. the operating table: what a threshold costs per day and what it buys
sk falsealarm report --scores 'scores_mant/6s/*.npz' --station MANT \
  --stations-csv catalogs/istasyon_katalog.csv \
  --catalog catalogs/catalog_current.csv \
  --window-seconds 6.0 --snr-csv mant_range.csv \
  --out-prefix mant_fa_6s

# 4. per event: when did it fire, relative to S
sk falsealarm timing --scores 'scores_mant/6s/*.npz' --station MANT \
  ... --window-seconds 6.0 --threshold 0.9971 --out mant_timing.csv

# 5. what requiring two stations to agree costs, and what it buys
sk falsealarm coincidence \
  --scores-a 'scores_mant/6s/*.npz' --station-a MANT \
  --scores-b 'scores_demi/6s/*.npz' --station-b DEMI \
  ... --window-seconds 6.0 --out-prefix coinc_mant_demi

# and a check that the scan path still matches the training pipeline
sk falsealarm verify --dataset-dir DS --ckpt-dir CKPT
```

Produces scores_<station>/<arm>/<chunk>.npz holding (t, p) per window, then threshold tables, event tables and coincidence tables as CSV.

--arm NAME:WINDOW:CKPT_DIR:BRANCH[:STEP] is repeatable, and every arm windows the same read – running two costs well under twice one, because reading and decoding the archive dominates. stalta as the CKPT_DIR slot runs a classical baseline on the identical stream.

Never inherit the benchmark threshold. The 6 s detector scores a median of 0.80 on continuous

noise and flags 92.7% of a quiet station-day at 0.5 – 12,599 alarms/day, against 257 extrapolated from benchmark FPR. `report` derives thresholds from the measured background instead; use those.

Scan announces before it reads. A fragmented chunk can take 17 minutes inside one opaque obspy call. Without that line a slow chunk is indistinguishable from a hang, which has already caused two false alarms of its own.

Windows are disjoint by default (step = window), so an alarm count is also a count of independent decisions. Pass a `:STEP` smaller than the window only for a dense rescan of event guards, via `--near-csv`, and expect ~12× more decisions per unit time at the same threshold.

OP 11 Reports and figures

```
sk docx in.md out.docx      # needs: uv run --with python-docx sk docx ...
sk pdf in.md out.pdf        # uv run --with markdown --with weasyprint sk pdf ...
sk figures                  # regenerates the report figures
```

sk figures ignores --help and does the work instead. There is no dry run; invoking it at all rewrites its output directory. Known bug.

The PDF stylesheet names `DejaVu Serif` first deliberately – the reports are in Turkish and the fallback stack must cover `ı`, `İ`, `ş`, `ğ` and `ç`.

OP 12 Find out what happened

```
sk status                  # running jobs, recent runs, free disk
sk status --host vegs      # the box the jobs are actually on

sk results                 # every run record, newest first
sk results --task magnitude
sk results --metric MAE --best
sk results --failed
sk results --show RUN
```

Every registry-wired trainer writes a record through `sismokaos.runlog : argv`, git commit

and whether the tree was dirty, dataset identity, split, seeds, metrics, checkpoints. A run that crashes leaves a record saying `started` rather than no record at all – so an empty `runs/` means nothing was launched through `sk train`, not that nothing ran.

sk results marks dirty-tree runs. A run from a dirty tree does not reproduce from its recorded commit; treat its number as provisional.